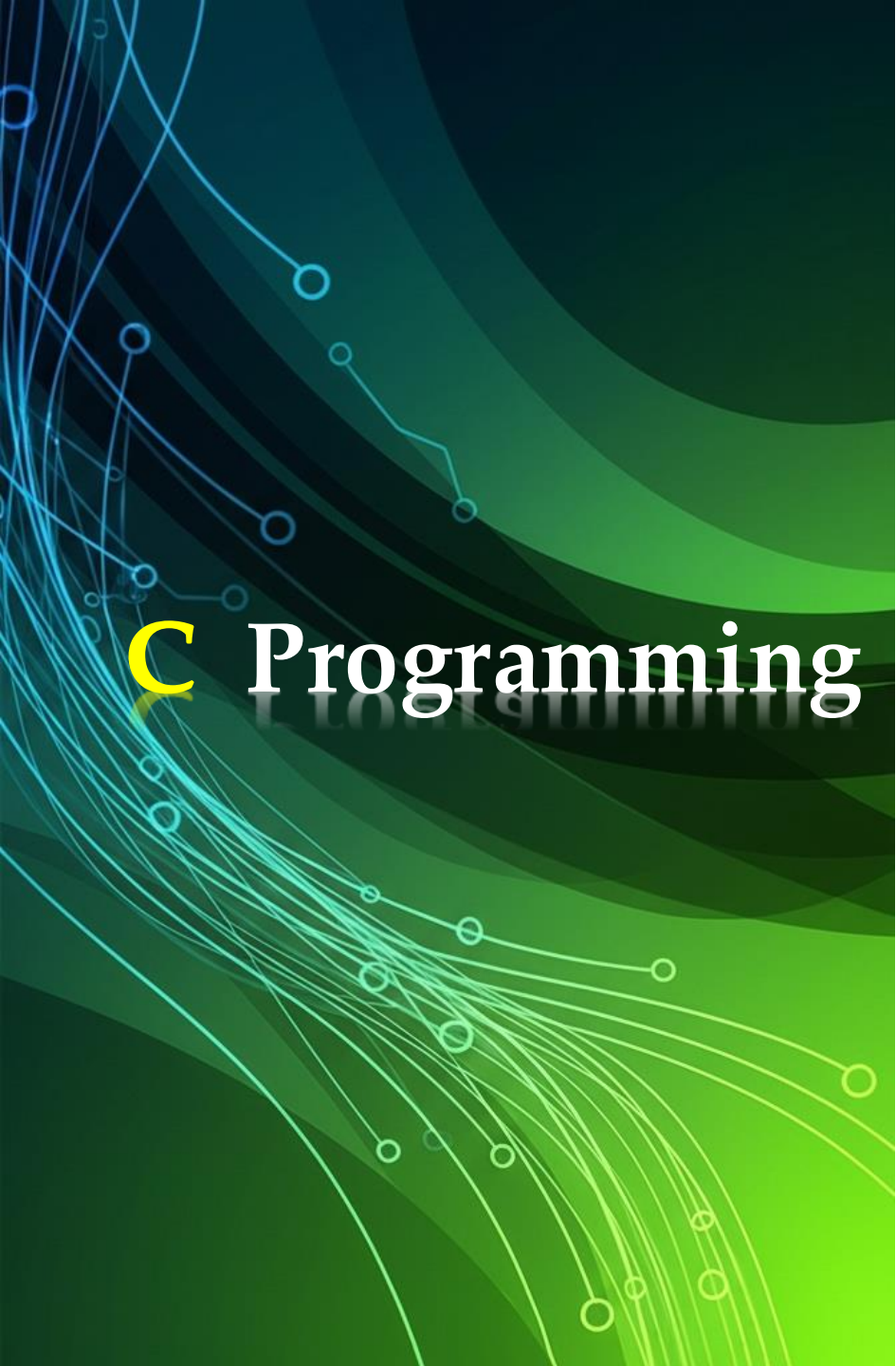




中山大学专业课《程序设计I》

## 第十二节 结构体进阶

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

# 猜一猜

node.y 的内存地址是?

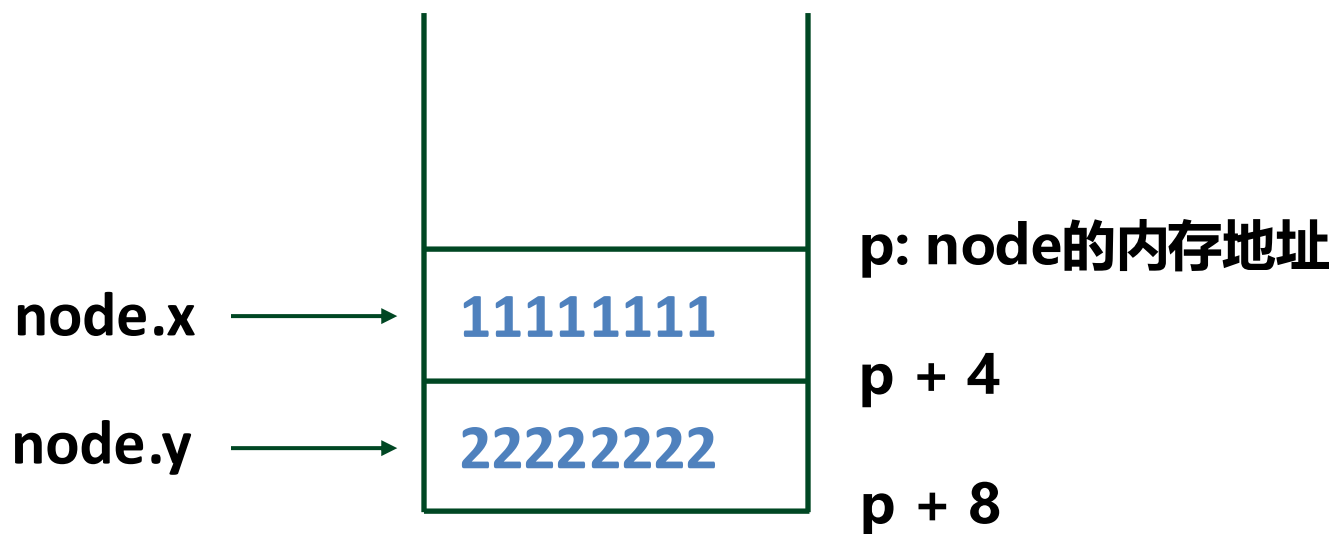
```
struct NODE {  
    int x, y;  
} node;  
  
printf("%p", &node.x);  
printf("%p", &node.y);  
printf("%zd", sizeof(node));
```



0x16fdfeeb4

# 结构体内存布局

在内存中，结构体实例变量分配在一段连续的内存空间内，每个成员变量依次被分配在后续的内存位置上



# 想一想

node.y 的内存地址是?

```
struct NODE {  
    char x;  
    int y;  
} node;  
  
printf("%p", &node.x);  
printf("%p", &node.y);  
printf("%zd", sizeof(node));
```

0x16fdfeeb4

# 字长 (word length) 与内存中数据

- ❑ 字长指CPU一次从内存读或写的位数，32 位字长表示一次读或写4字节
- ❑ 对于 32 位字长的CPU，每次访问内存的地址总是 4 的倍数

- 若将整数放置在非 4 倍的地址，如右图
- 则要么编译器用两条指令完成该整数读写并拼接为一个整数，要么硬件用两个以上微指令周期完成该整数读写
- 将导致CPU处理数据性能大幅下降

| 地址        | 值 (byte) |
|-----------|----------|
| 0x...0000 | 00       |
|           | 11       |
|           | 22       |
|           | 33       |
| 0x...0004 | 44       |
|           | 55       |
|           | 66       |

- ❑ 解决办法就是对齐 (alignment)

- 大于字长的基本类型数据都按字宽边界对齐，即使浪费一些内存也值得
- 因此，程序中整数变量地址一般是 4 (整数变量的对齐值) 的倍数

# 结构体成员的对齐

1. **成员对齐原则**：每个成员**相对于结构体起始地址**的偏移量（offset），需是该成员**自身大小（对齐值）**的整数倍
  - 若偏移量不满足，编译器会在成员之间**填充（padding）**空字节
2. **结构体整体对齐原则**：结构体的**地址和大小**，必须是其内部**最大基本类型成员大小（对齐值）**的整数倍
  - 如果不满足，编译器会在结构体的**末尾padding**
  - 这是为了确保在定义**结构体数组**时每一个元素都能满足规则 1

# 如何改变结构体的对齐值?

`#pragma pack(N)`

- 预编译指令 `#pragma pack(N)` 控制其后定义的和联合体的**最大对齐值**,  $N \in \{1, 2, 4, 8, 16, 32\}$ 
  - 基本数据类型**成员变量**对齐值规则:
    - 数据类型所占字节数  $\geq N$ , 则地址是  $N$  的倍数
    - $< N$  则对齐值是数据类型所占字节数的倍数
  - 结构体整体对齐规则:  $\min(\text{成员中对齐值最大值}, N)$ 
    - 除了地址是对齐值的倍数, 其大小也必须是对齐值的倍数

# 结构体对齐 – 案例

```
/*struct layout*/  
#pragma pack(1)  
  
typedef struct {  
    char a[3];  
    int b;  
    short c;  
} packed_struct;  
  
#pragma pack(4)  
  
typedef struct {  
    char a[3];  
    int b;  
    short c;  
} unpacked_struct;
```

| 地址  | 值 (byte) |
|-----|----------|
| a   | 00       |
| a+1 | 11       |
| a+2 | 22       |
| &b  | 33       |
|     | 44       |
|     | 55       |
|     | 66       |
| &c  | 77       |
|     | 88       |

N=1  
struct 对齐值 1  
struct size 9

| 地址 | 值 (byte) |
|----|----------|
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |
|    |          |

N=2  
struct 对齐值 ?  
struct size ?

| 地址  | 值 (byte) |
|-----|----------|
| a   | 00       |
| a+1 | 11       |
| a+2 | 22       |
|     | --       |
| &b  | 33       |
|     | 44       |
|     | 55       |
|     | 66       |
| &c  | 77       |
|     | 88       |
|     | --       |
|     | --       |

N=4  
struct 对齐值 4  
struct size 12

# 结构体对齐 – 案例

```
/*struct layout*/  
#pragma pack(1)  
  
typedef struct {  
    char a[3];  
    int b;  
    short c;  
} packed_struct;  
  
#pragma pack(4)  
  
typedef struct {  
    char a[3];  
    int b;  
    short c;  
} unpacked_struct;
```

| 地址  | 值 (byte) |
|-----|----------|
| a   | 00       |
| a+1 | 11       |
| a+2 | 22       |
| &b  | 33       |
|     | 44       |
|     | 55       |
|     | 66       |
| &c  | 77       |
|     | 88       |

N=1  
struct 对齐值 1  
struct size 9

| 地址  | 值 (byte) |
|-----|----------|
| a   | 00       |
| a+1 | 11       |
| a+2 | 22       |
|     | --       |
| &b  | 33       |
|     | 44       |
|     | 55       |
|     | 66       |
| &c  | 77       |
|     | 88       |

N=2  
struct 对齐值 2  
struct size 10

| 地址  | 值 (byte) |
|-----|----------|
| a   | 00       |
| a+1 | 11       |
| a+2 | 22       |
|     | --       |
| &b  | 33       |
|     | 44       |
|     | 55       |
|     | 66       |
| &c  | 77       |
|     | 88       |
|     | --       |
|     | --       |

N=4  
struct 对齐值 4  
struct size 12

# 位域 (bit field)

想象一下：你在开发一款大型游戏，内存中有**10万个小怪**，每个小怪都有各种状态（是否活着、是否中毒、是否在飞、是否隐身等）

```
struct MonsterStatus {  
    bool isAlive; // 占1字节  
    bool isPoisoned; // 占1字节  
    bool isFlying; // 占1字节  
    bool isStunned; // 占1字节  
};
```

**可能需要40万个字节！**

# 位域 (bit field)

想象一下：你在开发一款大型游戏，内存中有**10万个小怪**，每个小怪都有各种状态（是否活着、是否中毒、是否在飞、是否隐身等）

```
struct MonsterStatus {  
    bool isAlive    : 1; // 占用1位  
    bool isPoisoned : 1;    // 占用1位  
    bool isFlying   : 1; // 占用1位  
    bool isStunned  : 1;    // 占用1位  
};
```

**可能仅需5万个字节！**

# 位域

## 位域可以使得结构体支持方便的按位访问

```
struct A {  
    unsigned int t : 2;  
};
```



声明含义是，t 的位宽为 2 的无符号整数  
这里， $0 \leq \text{位宽} \leq \text{整数的位数}$

```
struct A a = { 2 };  
a.t = 3;  
a.t = 4; // warning: Implicit truncation from 'int'  
         // to bit-field changes value from 4 to 0
```

位域类型可以是整数，无符号整数，\_Bool

# 位域 – 对齐

- 如果几个位域被顺序写在一起，那么他们会被**打包拼合**在一起
  - 位域对齐值按整数对齐值，及第一个位域必须按整数对齐，一个包最多是一个整数

```
struct B {  
    unsigned t1: 10;  
    unsigned t2: 11;  
    unsigned t3: 11;  
};
```

```
printf("%lu\n", sizeof(struct B));
```

```
>>> 4
```



- 拼合的特性使得位域可以用于硬件寄存器一类的结构

# 位域 - 内存布局

匿名位域表示跳过若干位；0宽度位域表示当前整数包已满，下个位域分配在新整数单元；**位域不能求地址**，可与union配合获得地址

```
/*bit field layout*/
struct BF10 {
    int a:4;
} bit_field1;

struct BF11 {
    int a:4, :2, b:5;
} bit_field2;

struct BF20 {
    int a:4, :2;
    int b:27;
} bit_field3;

struct BF21 {
    int a:4, :2, :0;
    int b:4;
} bit_field4;
```

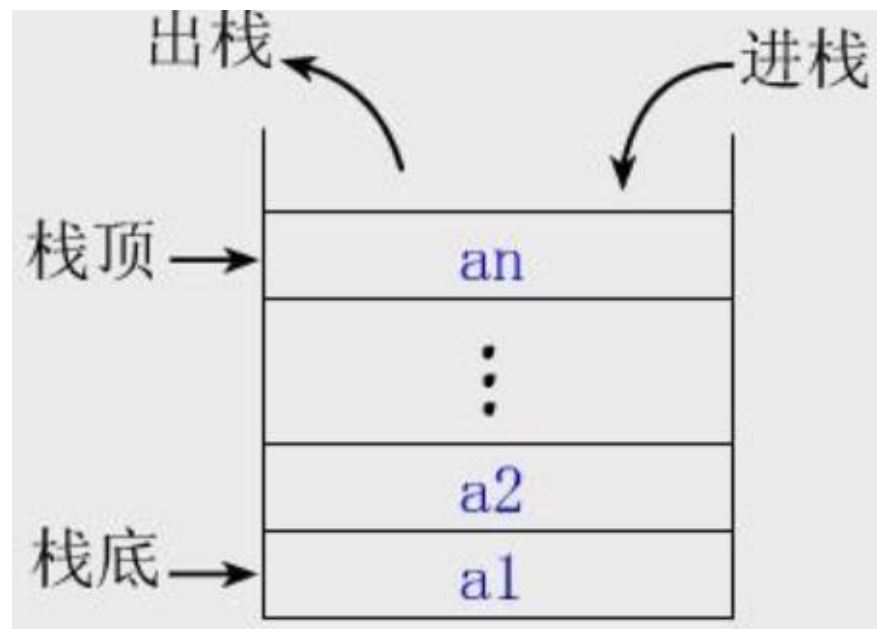
```
union {
    int i;
    struct BF11 bf11;
} u1 = {.i=0x5a}; // 0101 1010

union {
    int i[2];
    struct BF21 bf21;
} u2 = {.i={0x5a,0x5a}}; // 0101 1010

int main() {
    printf("bit_field1 size %d\n", sizeof(bit_field1));
    printf("bit_field2 size %d\n", sizeof(bit_field2));
    printf("bit_field3 size %d\n", sizeof(bit_field3));
    printf("bit_field4 size %d\n", sizeof(bit_field4));
    printf("%x,%x\n",u1.bf11.a, u1.bf11.b);
    printf("%x,%x\n",u2.bf21.a, u2.bf21.b);
}
```

# 栈 (stack)

- ❑ 栈是一种线性的数据结构，它只允许在一端进行插入或删除操作
- ❑ 它既可以用数组实现，也可以用链表实现



# 栈数据结构及应用

## 栈的基本操作

- 1) **void InitStack(Stack\* s): 初始化空栈S**
- 2) **int StackEmpty(const Stack\* s): 判断一个栈是否为空**
- 3) **void Push(Stack\* s, const int x): 进栈, 将x加入使之成为新栈顶**
- 4) **int Pop(Stack\* s): 出栈, 若栈非空, 则将栈顶元素返回**
- 5) **void DestroyStack(Stack\* s): 销毁栈, 并释放栈S占用的存储空间**

# 栈数据结构 - 链表实现

根据链表实现，可以完成栈的基本操作

// 定义栈

```
typedef struct StackList{  
    NODE_PTR head;  
} StackList;
```

// 入栈一个元素

```
void Push(StackList *s, const int x){  
    s->head=ListAdd(s->head,x);  
}
```

// 检查是否为空

```
int Pop(StackList *s){  
    assert(s->head);  
    int t=s->head->customerData;  
    s->head=ListRemove(s->head);  
    return t;  
}
```

# 栈数据结构 – 数组实现

使用数组也可以实现栈，由于数组是固定大小的，因此需要记录当前栈元素数量，以避免溢出

```
typedef struct StackArray{ // 定义栈
    int *data;
    int cnt; //记录目前栈内有多少元素
    size_t max_size; // 记录栈最多可以容纳多少元素
} StackArray;
```

```
void InitStack(StackArray* s,size_t size){ // 初始化栈
    s->data=(int*)malloc(size*sizeof(int));
    s->max_size=size;
    s->cnt=0;
}
```

# 栈数据结构 – 数组实现

使用数组也可以实现栈，由于数组是固定大小的，因此需要记录当前栈元素数量，以避免溢出

```
void Push(StackArray *s, const int x){ // 压栈
    assert(s->cnt < s->max_size);
    s->data[s->cnt] = x;
    s->cnt += 1;
}
```

```
int Pop(StackArray* s){ // 出栈
    assert(s->cnt);
    int t = s->data[s->cnt-1];
    s->cnt -= 1;
    return t;
}
```

# 练习：有效的括号 (LeetCode No. 20)

## 20. 有效的括号

简单

🔖 相关标签

🏢 相关企业

💡 提示

Aa

给定一个只包括 '('，')'，'{'，'}'，'['，']' 的字符串 `s`，判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

求解方法：从左到右扫描字符串，若遇到左括号入栈，遇到右括号时将栈顶的对应左括号出栈。若遇到右括号时栈顶的左括号与之不匹配，则无效；若到结束时栈内还有元素，则无效；其余情况皆为有效字符串

示例 2：

输入：s = "()[]{}"

输出：true

示例 3：

输入：s = "("

输出：false

示例 4：

输入：s = "([)]"

输出：true

示例 5：

输入：s = "([)]"

输出：false

# 练习：有效的括号

```
for(i=0;str[i]!='\0';i++){//从左到右扫描
    if(str[i]=='{' || str[i]=='(' || str[i]=='['){
        Push(&s,str[i]);//左括号直接入栈
    }else{//遇到右括号，则匹配对应括号出栈
        if(StackEmpty(&s)){//没有括号匹配，则字符串无效
            printf("false\n");
            return 0;
        }
        char c=Pop(&s);
        if((str[i]=='}' && c=='{') || (str[i]==')' && c=='(') || (str[i]==']' && c=='[')){
            continue;//括号匹配了，则继续
        }else{//括号不匹配，则字符串无效
            printf("false\n");
            return 0;
        }
    }
}
if(!StackEmpty(&s)){//最后栈非空则无效
    printf("false\n");
}else{
    printf("true\n");
}
```

# 练习：后缀表达式\*

| 表达式类型 | 运算符位置 | 举例 (计算 3加4再乘2)           | 计算机友好度         |
|-------|-------|--------------------------|----------------|
| 中缀    | 操作数中间 | <code>(3 + 4) * 2</code> | 低 (需处理括号/优先级)  |
| 后缀    | 操作数后面 | <code>3 4 + 2 *</code>   | 高 (最常用, 利用栈处理) |

**如何将中缀表达式转换为后缀表达式？**

# 练习：后缀表达式\*

调度场算法 (Shunting-yard Algorithm) 简述：

1. 遇数字：直接输出
2. 左括号 (：压入栈
3. 右括号 )：弹出栈中所有运算符直到碰到 (
4. 运算符 (+ - \* /): 弹出栈中优先级  $\geq$  自己的符号，然后入栈

# 练习：后缀表达式\*

$$a + b * (c - d) / e$$

| 步骤 | 读取符号 | 动作说明         | 栈 (底 → 顶) | 输出序列 (后缀表达式) |
|----|------|--------------|-----------|--------------|
| 1  | a    | 操作数，直接输出     | 空         | a            |
| 2  | +    | 栈空，直接入栈      | +         | a            |
| 3  | b    | 操作数，直接输出     | +         | a b          |
| 4  | *    | * 优先级高于 +，入栈 | + , *     | a b          |
| 5  | (    | 遇到左括号，直接入栈   | + , * , ( | a b          |
| 6  | c    | 操作数，直接输出     | + , * , ( | a b c        |

# 练习：后缀表达式\*

$$a + b * (c - d) / e$$

|    |    |                           |                  |                      |
|----|----|---------------------------|------------------|----------------------|
| 7  | -  | 栈顶是 (，优先级最低，- 直接入栈        | + , * , ( ,<br>- | a b c                |
| 8  | d  | 操作数，直接输出                  | + , * , ( ,<br>- | a b c d              |
| 9  | )  | 清算：弹出 - 输出，直到遇到 (         | + , *            | a b c d -            |
| 10 | /  | 栈顶 * 与 / 同级。弹出 * 输出， / 入栈 | + , /            | a b c d - *          |
| 11 | e  | 操作数，直接输出                  | + , /            | a b c d - * e        |
| 12 | 结束 | 扫描完成，将栈中剩余符号全部弹出          | 空                | a b c d - * e /<br>+ |

# 共用体

- 共用体与结构体一样是一种派生数据类型，在共用体中，成员共享一个存储空间

```
union number {  
    int x;  
    double y;  
};
```

- 例如number这个共用体中，x和y是共用一个存储空间的，共用体所占用的空间取决于对齐值

# 练习：IP地址表示

IP 地址是一个 unsigned long int，用于各种地址相关运算。出于可读性和便于记忆，通常用4个0-255的整数显示、编辑，例如：某机器 IP 地址是 172.6.6.24。

- 请给出 IP Address 的数据类型定义
- 并初始化一个 IP Address 的实例

# 练习：IP地址表示

```
union IPAdress {  
    unsigned long int full_ip;  
    struct {  
        unsigned char b1;  
        unsigned char b2;  
        unsigned char b3;  
        unsigned char b4;  
    } ip;  
};
```

```
union IPAdress my_ip = {  
    .ip = {172, 6, 6, 24}  
};  
printf("%d.%d.%d.%d\n",  
    my_ip.ip.b1,  
    my_ip.ip.b2,  
    my_ip.ip.b3,  
    my_ip.ip.b4);
```

# 谢谢！

邮箱: [yuesh5@mail.sysu.edu.cn](mailto:yuesh5@mail.sysu.edu.cn)